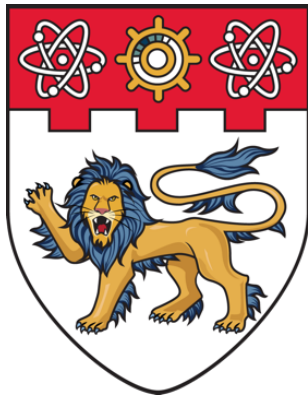




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

EE6483 GROUP PROJECT

Cats and Dogs

**TAN TIANQI G2404381H
XU LINHAN G2306155H
CHAN CHIN KIU G2404887L**

**NANYANG TECHNOLOGICAL UNIVERSITY
AY2425S1**

Contents

Contents

1	Introduction	1
2	Question a	2
3	Question b	3
3.1	Model Description	3
3.1.1	VGG-16	3
3.1.2	ResNet50	6
3.1.3	Vision Transformer-ViT	8
4	Question c	10
5	Question d	11
6	Question e	11
7	Question f	13
8	Question g	14
9	Question h	17
	References	18
A	File structure	19
B	Code	19

Contribution:

TAN TIANQI: Data preprocessing for the given dataset and all ResNet related parts including training, test and results analysis

XU LINHAN: Data preprocessing for the given dataset and all ViT related parts including training, test and results analysis.

CHAN CHIN KIU: Data preprocessing for the given dataset and all VGG-16 related parts including training, test and results analysis.

1 Introduction

Image classification is a cornerstone problem in computer vision, with applications spanning diverse fields such as healthcare, autonomous vehicles, and e-commerce. At its core, the task involves assigning an image to one of several predefined categories. The field has witnessed transformative progress, particularly with the advent of deep learning. However, the problem remains challenging under varying settings, such as supervised versus unsupervised learning, closed-set versus open-set classification, and scenarios involving domain shifts or imbalanced data.

Challenges and Solution Types

Supervised learning dominates image classification due to the availability of labeled datasets like ImageNet and CIFAR. Models like ResNet, VGG, and DenseNet have shown remarkable success. Unsupervised learning (e.g., clustering or contrastive learning) offers an alternative when labeled data is scarce, with models like SimCLR and MoCo gaining traction.

Closed-set classification assumes all test samples belong to known classes, simplifying the task. Open-set classification, however, handles "unknown" samples, introducing the need for rejection mechanisms, such as confidence scoring or generative approaches like OpenMax.

Domain Shift: Domain adaptation techniques, such as adversarial training (e.g., DANN) or feature alignment methods, address the challenge of transferring knowledge between source and target domains.

Imbalanced Data: Imbalanced datasets, where certain classes dominate, can bias models. Solutions include class reweighting, oversampling, or using robust architectures like attention mechanisms that focus on critical regions of the image.

Recent Advances and Multi-Model Approach

Traditional CNNs (e.g., ResNet, VGG): Convolutional Neural Networks (CNNs[1]) are widely used for their ability to learn hierarchical image features. ResNet introduced residual connections to alleviate gradient vanishing in deep networks, while VGG emphasizes simplicity in design but at higher computational costs.

Transformer Models (e.g., Vision Transformer, ViT): Vision Transformers[2] have emerged as a strong alternative to CNNs, processing images as patches and excelling in capturing global dependencies. These models achieve state-of-the-art results on several benchmarks but require extensive training data.

Hybrid Architectures: Combining CNNs and transformers (e.g., ConvNeXt, Swin Transformer) provides a balance between local feature extraction and global attention. Such architectures excel in scenarios requiring fine-grained and global feature understanding.

Self-Supervised Models: Methods like SimCLR[3], MoCo[4], and BYOL[5] pre-train models without labels by leveraging contrastive or predictive tasks. These models fine-tune well on small labeled datasets and are effective in low-resource scenarios.

Few-Shot and Zero-Shot Learning: Methods like Prototypical Networks and CLIP[6] address few-shot or zero-shot tasks, leveraging pre-trained embeddings or textual prompts to generalize to unseen categories.

2 Question a

The original dataset consists of 20,000 training samples, 5,000 validation samples, and 500 test samples. The distribution of classes is shown in the table1 below:

Table 1: The distribution of Raw Dataset

	Cats	Dogs	Sum
Training Data	10000	10000	20000
Validation Data	2500	2500	5000
Testing Data	241	259	500

Data Preprocessing

We firstly normalize the input RGB values to $[0, 1]$ by rescaling since the original data is simply too high for a model to process.

Data Augmentation

Data augmentation is a prominent process to generate more relevant samples and increase the size of training set to make the model more robust along with reduce overfitting of the model.

We apply Albumentations, a Python library specifically used for image augmentations. The specifically techniques are shown as the table2 below.

Table 2: Data augmentation measures taken for the training set

Techniques	Description
ChannelShuffle	Randomly rearrange channels of the RGB image($p = 0.5$)
Hue Saturation Value	Randomly change hue, saturation and value of the image($p = 0.5$)
RandomRotate	Randomly rotate the image by 90 degrees zero or more times($p = 0.5$)
ShiftScaleRotate	Randomly apply translate, scale and rotate the image($p = 0.75$)
Blur	Blur the input image using a random-sized kernel(kernel.size = 5, $p = 0.5$)
RandomBrightnessContrast	Randomly changes the brightness and contrast of the input image($p = 0.5$)
HorizontalFlip	Randomly flips the image horizontally($p = 0.5$)
VerticalFlip	Randomly flips the image vertically($p = 0.5$)

Through the measures mentioned above, we finally generate a total of 40,000 images of the training set, consisting of 20,000 raw images and 20,000 newly images. We use Albumentation's Compose function to build augmentation pipes with a random seed of 10 to make sure the result is reproducible. As far as concerned with the validation dataset, we decide not to do the augmentation since we consider its size is enough. Noted that data amplification has its limitations, i.e. there are still biases inherent in the original data. We also keep the balance between dogs and cats in both the training and validation datasets. Therefore, the model is not biased in favour of the majority class, resulting in lower accuracy for the minority class.

3 Question b

For the Dogs vs. Cats problem, we build three different models to construct our classifiers.

3.1 Model Description

3.1.1 VGG-16

The first model that we utilized is the VGG-16 by transfer learning. VGG-16 is a Convolutional Neural Network (CNN) architecture that was introduced by Simonyan and Zisserman in their 2014 paper, "Very Deep Convolutional Networks for Large-Scale Image Recognition." [7] It is one of the most widely used models for image classification and feature extraction, known for its simplicity and effectiveness.

The VGG-16 network has the following characteristics: depth, uniform design, activation function, fully connected layers, and pretrained weights. The model consists of 16 weight layers: 13 convolutional layers and 3 fully connected (dense) layers. It adopts a simple and uniform architecture, stacking convolutional layers with small 3x3 kernels rather than using larger filters like 5x5 or 7x7. This increases depth while maintaining manageable parameters. The activation function used for every layer is the Rectified Linear Unit (ReLU), which introduces non-linearity. Finally, there are pretrained weights for this model which allows us to do transfer learning for this implementation.

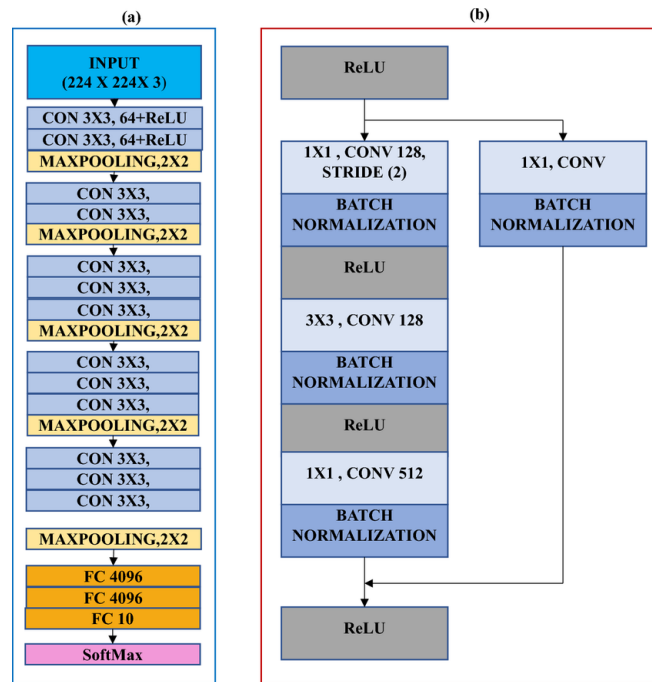


Figure 1: (a) VGG-16 Architecture (b) Residual Block

Following this model architecture, we have the following layers in our model with the number of parameters in 2 and 3.

Using transfer learning, we will be training the last 5 layers of the model: flatten, Dropout1, dense, Dropout2, and dense_1.

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 370, 370, 3)	0
block1_conv1 (Conv2D)	(None, 370, 370, 64)	1,792
block1_conv2 (Conv2D)	(None, 370, 370, 64)	36,928
block1_pool (MaxPooling2D)	(None, 185, 185, 64)	0
block2_conv1 (Conv2D)	(None, 185, 185, 128)	73,856
block2_conv2 (Conv2D)	(None, 185, 185, 128)	147,584
block2_pool (MaxPooling2D)	(None, 92, 92, 128)	0
block3_conv1 (Conv2D)	(None, 92, 92, 256)	295,168
block3_conv2 (Conv2D)	(None, 92, 92, 256)	590,080
block3_conv3 (Conv2D)	(None, 92, 92, 256)	590,080
block3_pool (MaxPooling2D)	(None, 46, 46, 256)	0
block4_conv1 (Conv2D)	(None, 46, 46, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 46, 46, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 46, 46, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 23, 23, 512)	0
block5_conv1 (Conv2D)	(None, 23, 23, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 23, 23, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 23, 23, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 11, 11, 512)	0
flatten (Flatten)	(None, 61952)	0
Dropout1 (Dropout)	(None, 61952)	0
dense (Dense)	(None, 128)	7,929,984
Dropout2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

Figure 2: VGG-16 Layers and Params

```

Total params: 22,644,930 (86.38 MB)

Trainable params: 7,930,242 (30.25 MB)

Non-trainable params: 14,714,688 (56.13 MB)

```

Figure 3: Number of trainable and non trainable parameters

The optimizer that we will be using for implementation is Adam (Adaptive Moment Estimation). It is an optimization algorithm that combines the benefits of momentum-based optimization and adaptive learning rates. Some of its key features include computing individual learning rates for each parameter, using exponential moving averages for gradients (momentum) and squared gradients (RMSprop). The reason why we chose the Adam optimizer is because it is one of the most efficient and widely used optimizers currently for deep learning models.

Next the loss function that we chose for the model is Categorical Crossentropy. It is used for multi-class classification problems where the labels are one-hot encoded, which applies to our data format. The loss function measures the difference between the predicted probability distribution (softmax output) and the true probability distribution (ground truth). The loss function can be given as:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Where y_i is the true label for class i while $\hat{y}_i$ is the predicted probability for class i .

Finally, we will be implementing two callbacks for our model: ReduceLROnPlateau and EarlyStopping. The reason why we use callbacks is because they are mechanisms to monitor training and dynamically adjust the training process. ReduceLROnPlateau is a learning rate reduction process where it will dynamically reduce the learning rate when the validation accuracy stops improving. It also helps the optimizer converge better by reducing the learning rate when the model approaches a plateau in performance. EarlyStopping is a callback function that stops training early if the model's validation loss does not improve for a defined number of epochs. It helps to prevent overfitting by stopping training when the model starts to degrade on the validation set. We chose a patience of five, which meant that the training will stop if validation loss does not improve for five consecutive epochs.

Table 3: Results

	acc	loss	val_acc	val_loss	learning_rate
0	0.964550	0.701553	0.974800	0.119315	0.001000
1	0.987750	0.041040	0.978600	0.063200	0.001000
2	0.991300	0.024683	0.980400	0.062100	0.001000
3	0.992250	0.028320	0.980200	0.092009	0.001000
4	0.994150	0.017899	0.981000	0.080023	0.001000
5	0.995250	0.014875	0.979200	0.089158	0.001000
6	0.994700	0.018117	0.982600	0.102659	0.001000
7	0.994600	0.021079	0.980000	0.165053	0.001000

Using the VGG-16 model, we were able to achieve a validation accuracy of over 98% as shown in 4. The training stopped after 7 epochs as the validation loss was minimum at the second epoch.

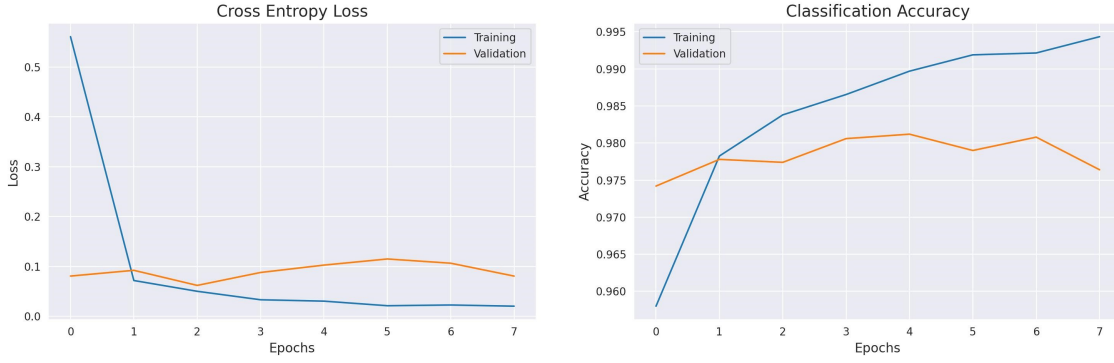


Figure 4: Cross Entropy Loss and Classification Accuracy Graphs

3.1.2 ResNet50

We also tried ResNet50[1] by transfer learning. Transfer learning is a machine learning and deep learning technique that allows us to accelerate model training and improve performance by transferring knowledge or features learned from one task to another related task. In transfer learning, we typically take a model that has been pre-trained on a large-scale dataset (named the source task model) and use its weights for a new task (the target task), rather than training a completely new model from scratch. Considering the classification mission, we used the ResNet50 that was pretrained on ImageNet.

ResNet50 is a deep residual network, proposed by Kaiming He et al. at Microsoft Research in 2015. It contains 50 convolutional layers and solves the problem of gradient vanishing and gradient explosion in deep networks by introducing Residual Blocks and Skip Connection. The network structure of ResNet50 can be divided into multiple stages from input to output, and each stage consists of multiple Residual Blocks that extract features by Each stage consists of multiple residual blocks that are stacked with convolutional kernels of different sizes to extract features. The network structure and parameter settings of ResNet50 allow it to learn complex features from data, enabling highly accurate image classification and target detection. The following image6 shows the ResNet50 model architecture.

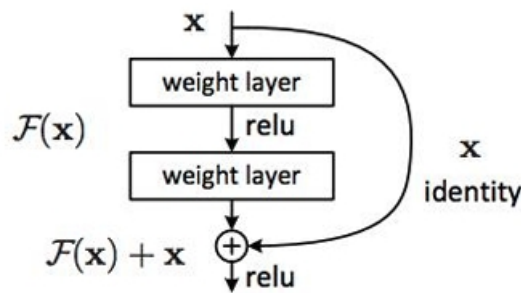


Figure 5: Residual block

Therefore, our model consists of three types of layers as shown in Table4.

Loss Function:

The binary cross entropy loss function was used for this task considering the binary image classification. It measures the difference between the model's predicted probabilities and the actual probabilities. When the model outputs a probability value, the Binary Cross Entropy Loss Function helps us understand the

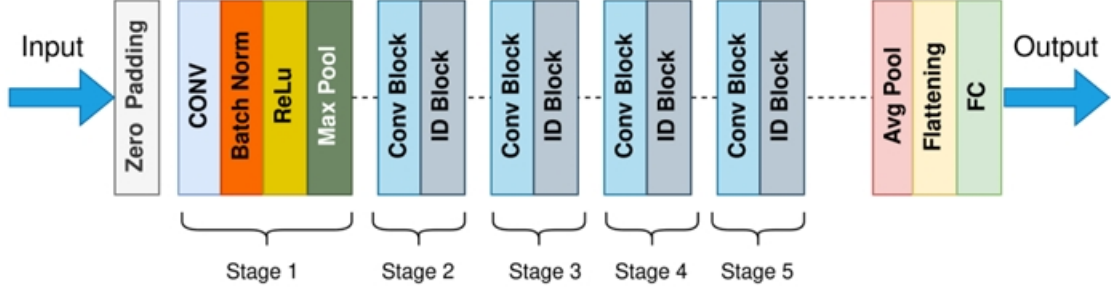


Figure 6: ResNet50 model architecture

Table 4: Model Architecture

Layer(type)	Output shape	Param
Rescaling Layer	(128, 128, 3)	0
ResNet50	(2048)	23,587,712
Dense Layer	(1)	2,049
Total params: 70,663,045		
Trainable params:23,536,641		
Non-trainable params:53,120		
Optimizer params:47,073,284		

accuracy of the model's predictions. The formula for the Binary Cross Entropy Loss Function is as follows:

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)] \quad (1)$$

where y is the true label, which takes a value of 0 or 1. p is the probability predicted by the model for the positive class. $\log$ is the natural logarithm.

The Binary Cross Entropy Loss Function is minimized during the training process to make the model's predicted probabilities as close as possible to the true labels.

Optimizer:

Adam is applied on the model. This is an adaptive optimiser that updates the learning rate based on the average first moment of the gradient (the mean) and the average of the second moments (the uncentred variance). the learning rate is the average of the first moments of the gradient (the mean) and the second moments of the gradient (the uncentred variance). As for the learning rate, we used 0.001 as the initial learning rate since we found it to be small enough to converge and large enough so that it did not require relatively long computation time.

Epochs and Early Stopping:

Early stopping strategy is used during the learning procedure. Its core idea is to stop training the model when its performance on the validation set no longer improves during the training process of the model. This not only saves computational resources, but also avoids overfitting the model on the training set, which improves the model's generalisation ability on unknown data. We set the patience (number of epochs it will wait) to be 10. The initial number of epochs was set to be 50 so that the model can converge until it ends(our model has training rounds ranging from 17 to 34 epochs according to our

training history).

Accuracy and Losses:

As the figure7 shown below, we achieved an accuracy of 90.6% in the validation dataset using this model architecture after stopped by the early stopping mechanism. Image8 showcases the loss function curve during the training process with the loss of 0.3697 in validation dataset.

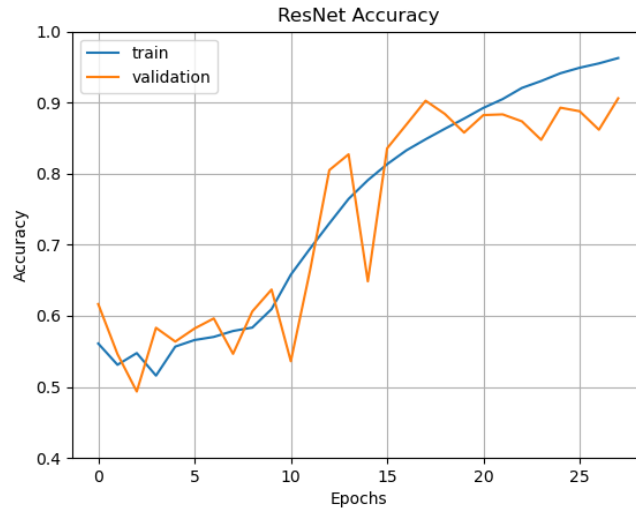


Figure 7: Resnet50 Accuracy

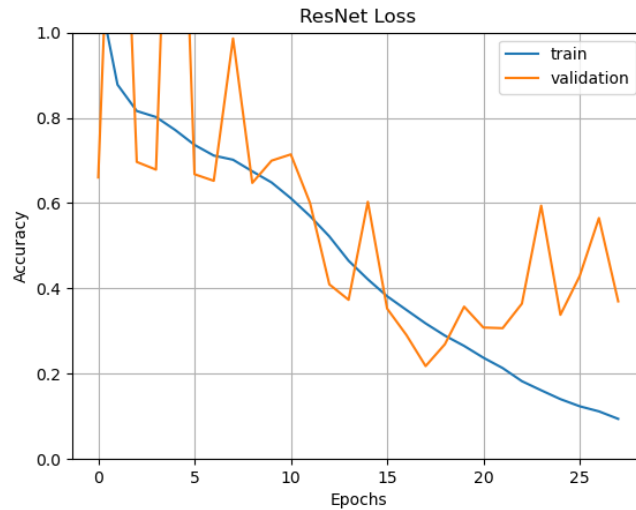


Figure 8: Resnet50 Loss

3.1.3 Vision Transformer-ViT

Vision Transformer (ViT) is a neural network architecture for processing images. It draws on the Transformer structure originally used to process sequence data.

ViT divides the image into a series of small blocks, each of which is linearly embedded into a flat

one-dimensional vector and adds position information, which ensures that the model can understand the relative or absolute position of the image block in the original image. After inputting the serialized image blocks, ViT uses the self-attention mechanism to capture the relationship between each image block, thereby processing the image information globally.

The figure9 shows the basic idea and structure of ViT.

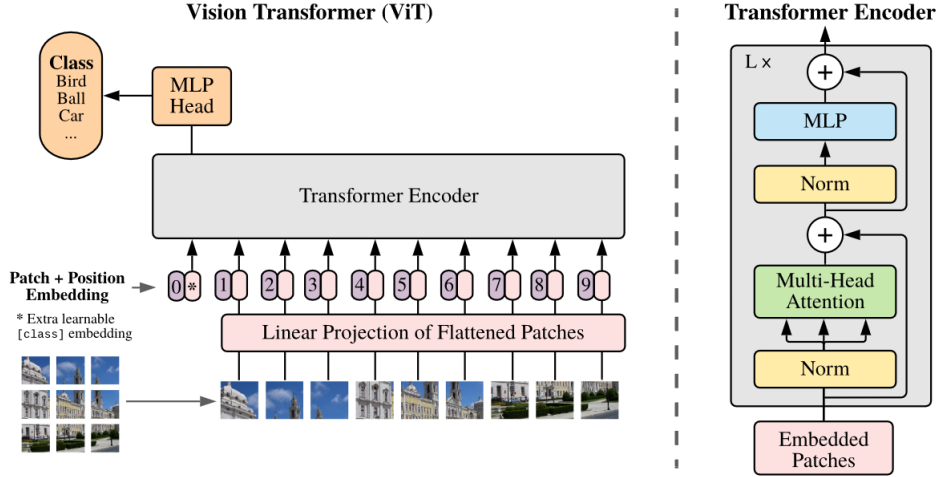


Figure 9: Vision Transformer-ViT model overview

ViT uses the encoder in Transformer for superposition, but the number of parameters is different. The token [197,768] with position encoding and special characters cls is passed to the encoder, first through layer normalization, and then through multi-head self-attention. The multi-head self-attention here uses 12 heads, that is, the 768 dimensions are divided into 12 parts, each part (Q, K, V) has 64 dimensions, and then merged into 768 dimensions after calculation.

The encoder in ViT still uses residual connection, and after another layer normalization, it enters the last layer MLP (multi-layer perceptron) of a single Transformer Encoder. MLP increases the output dimension of the multi-head self-attention by 4 times, that is, from 768 to 3072, and finally reduces the dimension to 768 dimensions.

After passing through the L-block Transformer Encoder module, the special character cls introduced later is used as the feature output of the entire network, which is equivalent to the feature after passing through n convolutional layers. Finally, the probability output can be obtained through the fully connected layer and softmax.

Model	Layers	Hidden Size D	MLP Size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 5: Details of Vision Transformer model variants.

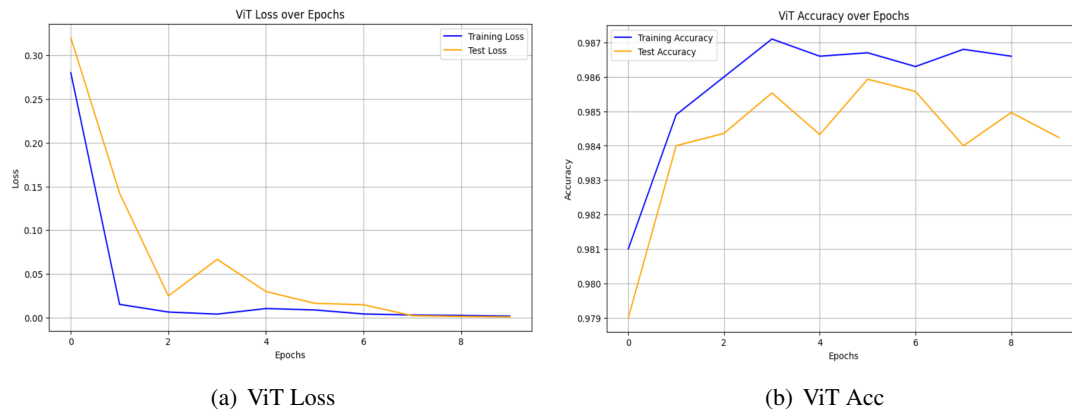


Figure 10: ViT accuracy and loss

The ViT model has shown effectiveness in multiple image recognition tasks (especially training on large-scale datasets). However, ViT has a high computational cost and may not be the best choice when real-time or resource-constrained conditions are required.

The figure10 shows the training accuracy and loss curves of each epoch on the cats and dogs dataset.

Code running:

The training code of the ViT model is the Vision-Transformer-ViT project file in the deliverable. Run the main function for training and validation. For specific parameters, see the settings in the argparse module.

4 Question c

1. VGG-16

Learning Rate (0.001): A learning rate of 0.001 was chosen because it is a good rate at which the trade-off between convergence speed and stability is optimized for the Adam optimizer.

Batch Size (64): A batch size of 64 was used as we utilized transfer learning which meant less parameters needed to be updated and the hardware used to train the model was able to keep up with the memory requirements. This led to a faster training without sacrificing computing power.

Image Size (370x370): By evaluating the average image size of our dataset, we determined that 370x370 was the average image size. This allowed for us to input the data without having to compress or expand the image too much, leading to less noise.

Early Stopping (patience=5): In order to prevent overfitting of the model to the training data, we employed the early stopping callback to end the training early when validation loss plateaus. We chose a patience of 5 here as the model converged pretty quickly with a batch size of 64.

Loss Function(Categorical Cross Entropy): We chose the categorical cross entropy loss function as it is a more general loss function which also applies to binary classification. It is also helpful when training the same model on the CIFAR-10 dataset in future sections.

2. ResNet50

Learning Rate (0.001): A learning rate of 0.001 was chosen as it is a commonly used starting point for the Adam optimizer, providing a balance between convergence speed and stability.

Batch Size (32): A batch size of 32 was selected because it offers a good trade-off between memory usage and model convergence. Larger batch sizes may require more memory, while smaller batch sizes can lead to noisier gradient updates.

Image Size (128x128): The input image size of 128x128 was determined based on the computational limits and the ResNet50 model's requirements. ResNet50 can handle variable image sizes but reducing the resolution ensures faster processing while retaining sufficient details for classification.

Early Stopping (patience=10): Early stopping with a patience of 10 epochs prevents overfitting by halting training when the validation loss does not improve for 10 consecutive epochs. This balances model training time with generalization performance.

Base Model (ResNet50 with ImageNet Weights): ResNet50 was chosen as the base model for its strong performance in feature extraction tasks. Pretrained weights on ImageNet help transfer learning, leveraging general image features to speed up convergence and improve results.

Loss Function (Binary Crossentropy): Binary crossentropy was selected as it is the standard loss function for binary classification tasks, ensuring an effective measure of prediction error for this problem.

3. ViT

The following are the best parameters selected during the training of ViT. 1. learning_rate: The learning rate is set to 2e-5 2. beta1: beta1 parameter of Adam optimizer (0.99). 3. beta2: beta2 parameter of Adam optimizer (0.99). 4. weight_decay: Weight decay value (default value is 2e-5). 5. batch_size: The number of data in each batch is 64

Model parameters of ViT:

1. emb_dim: The dimension of the embedding vector is 768 2. mlp_dim: The hidden layer dimension of the MLP layer is 3072 3. num_heads: The number of heads of the attention mechanism is 12 4. num_layers: The number of Transformer layers is 12 5. attn_dropout_rate: The dropout rate of the attention layer is 0.0. 6. dropout_rate: Dropout is 0.1. Randomly turn off some neurons to avoid overfitting.

5 Question d

Table 6: Classification accuracy on validation set from different models

Model	Training Accuracy	Validation Accuracy
VGG-16	99.5%	98.3%
Resnet50	95.31%	90.6%
VIT	98.7%	98.4%

CSVs from different models are attached.(Considering the results gained by Resnet50 are overfit, the result on the test data is gained through the Resnet13)

6 Question e

Using the VGG-16 model and predicting the results of the test data, we get the following classification report below:

Looking at the confusion matrix, we can see that there are a total of 9 misclassified samples, 7 samples for dog and 2 samples for cat. The resulting error rate is 98.2%.

	precision	recall	f1-score	support
Cat	0.97	0.99	0.98	241
Dog	0.99	0.97	0.98	259
accuracy			0.98	500
macro avg	0.98	0.98	0.98	500
weighted avg	0.98	0.98	0.98	500

Figure 11: Classification report on test data for VGG-16 model

Confusion Matrix

	Cat	Dog
True Label		
Cat	239	2
Dog	7	252
	Cat	Dog
	Predicted Label	

Figure 12: Confusion matrix on test data for VGG-16 model

Due to ViT's block processing of images, the complex background of the image may cause incorrect judgments. In particular, in the whole picture, the cat is among the cluttered objects, has a small proportion and is partially blocked by the background.

The cat & dog dataset used in this project is a small-scale dataset, and ViT may not be as effective as a convolutional neural network (CNN) designed to capture local structures of images. ViT may not be able to identify small differences in sample images. For example, the difference between dogs and cats of certain colors is small only from the back.

Disadvantages of the ViT:

In vision tasks, CNN exploits the local structure and translation invariance of images. Compared with CNN, ViT relies more on the global attention mechanism and lacks this prior structure of CNN, so it may not perform as well as CNN on small datasets.

Another problem is that a small dataset may not be enough to train an effective ViT model, because ViT requires a lot of data to learn global dependencies. On the other hand, training ViT on a small dataset is prone to overfitting, because the model may remember the training data instead of learning generalized features.

For the Resnet model:

Strength: Feature extraction: ResNet-13 effectively captures distinct features such as the dog's ears, facial structure, and texture due to its hierarchical feature extraction layers. Clear boundaries: The model performs well when the subject is clearly distinguishable from the background. Balanced dataset training: The ability to generalize to typical dog features shows that the model has learned a strong representation of this class. Case: A blurred image of a cat in a cluttered background with low contrast. Weakness: Background confusion: The cluttered environment may lead to misclassification as the model confuses background features with those of the opposite class. Blurriness sensitivity: The model

struggles with images where fine details, such as fur texture or facial features, are not prominent due to motion blur or low-quality resolution. Dataset limitation: If the training data lacks sufficient examples of low-quality or cluttered images, the model's performance drops in such scenarios.

7 Question f

1. Model Selection:

VGG16: VGG16 has a straightforward architecture with deep convolutional layers but relatively few optimizations like skip connections. It performs well on small datasets due to its simplicity but might struggle with overfitting or computational inefficiency. Validation accuracy is often reasonable but might plateau compared to more modern architectures. **Strength:** Easy to fine-tune, reliable for small datasets. **Weakness:** High computational cost, prone to overfitting. **ViT (Vision Transformer):** ViT processes images as sequences of patches, excelling in capturing global relationships across the entire image. However, it requires more training data and careful data augmentation to achieve optimal performance. On smaller datasets, ViT may underperform compared to CNNs unless pre-trained weights are used. **Strength:** Excellent feature representation on large datasets. **Weakness:** Sensitive to smaller datasets and insufficient augmentations.

ResNet50: ResNet50 utilizes skip connections to alleviate vanishing gradient problems, enabling effective training of deeper networks. It provides a balance between computational efficiency and accuracy, often achieving high validation accuracy on medium-sized datasets. **Strength:** Robust and versatile, performs well with moderate training data. **Weakness:** Can be less effective if images lack hierarchical feature structures.

Transfer learning using a pretrained model could also yield optimal results. However, in our case, we saw overfitting on the ResNet50 model. - For transfer learning, one should use a model that was pretrained on a similar dataset. If the selected model is too large, it might overfit very quickly, which will lower the validation accuracy.

ViT: For the ViT model, we compared the effects before and after using the pre-trained model. After using the pre-trained model, the accuracy of the test set increased from 92.0% to about 98.4%, which is a significant improvement.

Using the method of pre-training on a large-scale dataset, the ViT model is pre-trained and then fine-tuned on a small dataset. This method can help the model obtain useful feature representations, thereby reducing dependence on small datasets. (The ViT model was pretrained on ImageNet-21k, a dataset consisting of 14 million images and 21k classes.)

2. Data Processing:

Normalization: Standardizing pixel values (e.g., scaling to [0, 1]) ensures consistent input to the models, speeding up convergence and improving validation accuracy.

Data Augmentation: Augmenting training data (e.g., flips, rotations, and brightness changes) reduces overfitting and improves model generalization, leading to higher validation accuracy. This is especially important for ViT due to its data-hungry nature.

Image Resizing: Resizing all images to a fixed size (e.g., 128x128 or 224x224) is crucial for models like VGG16 and ResNet50, which require consistent input dimensions. Improper resizing can lead to loss of important features, reducing validation accuracy.

8 Question g

Dataset Details:

CIFAR-10 consists of 60,000 32x32 color images in 10 classes, with 6,000 images per class. The dataset is divided into 50,000 training images and 10,000 testing images. Each class (e.g., airplane, car, bird, etc.) is evenly distributed, making this a balanced, multi-category classification problem. Problem: This is a multi-class classification task, where the goal is to assign one of the 10 class labels to each image. Unlike the binary classification problem in Dogs vs. Cats, this task requires handling more complex feature representations and ensuring the algorithm generalizes well across all 10 classes.

1.VGG-16:

For the VGG-16 model, when training and fitting to the CIFAR-10 dataset, we were able to achieve a validation accuracy of 62.34% after training for 39 epochs. The validation loss and accuracy graphs are shown in Figure13:

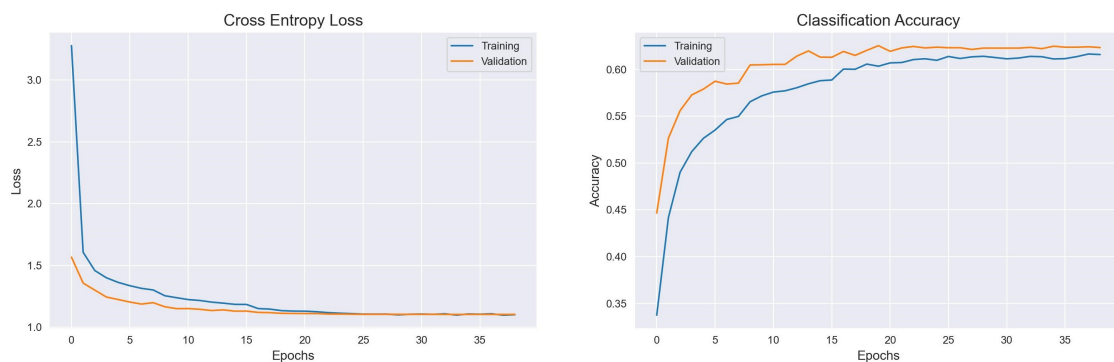


Figure 13: Validation loss and accuracy for VGG-16 on CIFAR-10 dataset

The table below shows the classification report for the model in 7.

The confusion matrix is also shown with the correct and incorrect classifications in 14.

		Confusion Matrix									
True Label	Airplane	175	4	13	4	4	1	4	10	25	14
	Automobile	5	174	3	13	1	1	6	3	8	36
	bird	18	5	126	15	23	11	19	10	5	1
	cat	8	5	18	134	11	32	24	10	6	12
	deer	9	6	17	7	148	10	23	24	6	5
	dog	1	8	13	60	9	127	11	14	0	3
	frog	1	6	15	20	19	8	182	2	4	2
	horse	6	2	10	14	25	15	3	167	4	8
	ship	22	19	3	6	5	3	7	5	185	14
	truck	9	28	1	8	3	0	1	4	9	157
		Predicted Label									

Figure 14: Confusion Matrix for VGG-16 on CIFAR-10 dataset

2.Resnet-13:

ResNet13 is a simplified ResNet model with 13 layers, leveraging residual connections to tackle the vanishing gradient problem and improve training stability. Residual blocks allow deeper models to converge

Table 7: Classification Report for VGG-16

	precision	recall	f1-score	support
Airplane	0.688976	0.688976	0.688976	254.000000
Automobile	0.677043	0.696000	0.686391	250.000000
bird	0.575342	0.540773	0.557522	233.000000
cat	0.476868	0.515385	0.495379	260.000000
deer	0.596774	0.580392	0.588469	255.000000
dog	0.610577	0.516260	0.559471	246.000000
frog	0.650000	0.702703	0.675325	259.000000
horse	0.670683	0.657480	0.664016	254.000000
ship	0.734127	0.687732	0.710173	269.000000
truck	0.623016	0.713636	0.665254	220.000000
accuracy			0.630000	2500.000000
macro avg	0.630341	0.629934	0.629098	2500.000000
weighted avg	0.631171	0.630000	0.629580	2500.000000

faster and achieve better performance on image data. Preprocessing:

For CIFAR-10: Resize images to 32x32 (native size). Normalize pixel values to [0, 1]. Perform data augmentation, including random cropping, flipping, and rotation, to increase training data diversity. Compared to Dogs vs. Cats, no resizing to larger dimensions (e.g., 224x224) is needed, as CIFAR-10 images are already small. Changes from Dogs vs. Cats Problem:

Output Layer: Adjust the output layer to Dense(10, activation='softmax') for multi-class classification instead of Dense(1, activation='sigmoid') for binary classification. Loss Function: Use SparseCategoricalCrossentropy or CategoricalCrossentropy to handle multi-class labels, replacing the binary cross-entropy used in binary classification. Metrics: Use accuracy as the evaluation metric, adjusted for multi-class tasks. Augmentation: Stronger augmentation is applied due to the smaller image size and lower resolution in CIFAR-10.

The accuracy for the training set is 85.3% and 82.7% for the validation set. The prediction for the test data is shown in Figure15.

3. ViT

In the second part of the project, we use the CIFAR-10 dataset. Compared with the dataset for cat and dog classification in this report, the CIFAR-10 dataset is slightly larger in size. Compared with the more complex datasets generally used for pre-training ViT models, the CIFAR-10 dataset is small and low-resolution.

In terms of data preprocessing, we first normalize the data: Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) to normalize each channel. We then resize the input image to 224x224 and convert the image into a tensor.

Table 8 shows the classification report of the ViT model after 25 epochs of training, achieving an accuracy of 98.57%.The two categories with the worst performance are 3 and 5, with an F1 score of about 96%.

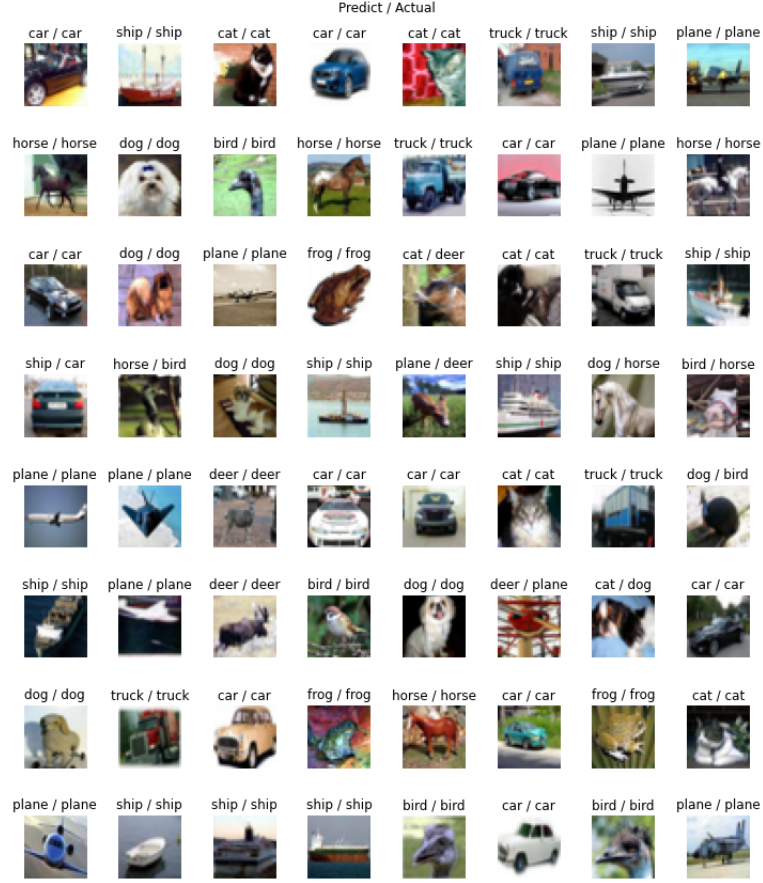


Figure 15: Predictions on the test set for Resnet-13

Table 8: Classification Report for ViT

Class	Precision	Recall	F1-score	Support
0	0.995968	0.9880	0.991968	1000.0000
1	0.985030	0.9870	0.986014	1000.0000
2	0.990010	0.9910	0.990505	1000.0000
3	0.969636	0.9580	0.963783	1000.0000
4	0.988945	0.9840	0.986466	1000.0000
5	0.963439	0.9750	0.969185	1000.0000
6	0.995025	1.0000	0.997506	1000.0000
7	0.994990	0.9930	0.993994	1000.0000
8	0.986152	0.9970	0.991546	1000.0000
9	0.987952	0.9840	0.985972	1000.0000
accuracy	0.985700	0.9857	0.985700	0.9857
macro avg	0.985715	0.9857	0.985694	10000.0000
weighted avg	0.985715	0.9857	0.985694	10000.0000

9 Question h

To address the issue of class imbalance in the CIFAR-10 dataset, two effective approaches are:

1. **Class Weight Adjustment** Assign higher weights to minority classes during loss calculation. This ensures that errors in these underrepresented classes contribute more to the total loss, thereby increasing the model's attention to these classes. When the dataset is imbalanced, the model naturally favors the majority classes because their samples dominate the training. Adjusting class weights ensures that minority classes are treated equally important, improving overall accuracy and fairness.

2. **Data Augmentation for Minority Classes** Apply data augmentation techniques (e.g., flipping, rotating, shifting) to artificially increase the number of samples for underrepresented classes. This balances the dataset by enhancing the representation of minority classes. Data augmentation generates new samples by applying transformations to existing ones, preserving the original class labels. This increases the diversity of training data, which helps the model generalize better to unseen samples. Minority classes often lack sufficient data for the model to learn their features effectively. Augmentation improves the model's ability to recognize these features by exposing it to a larger variety of samples from the minority classes.

References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [2] Alexander Kolesnikov, Alexey Dosovitskiy, Dirk Weissenborn, Georg Heigold, Jakob Uszkoreit, Lucas Beyer, Matthias Minderer, Mostafa Dehghani, Neil Houlsby, Sylvain Gelly, Thomas Unterthiner, and Xiaohua Zhai. An image is worth 16x16 words: Transformers for image recognition at scale. 2021.
- [3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- [4] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9729–9738, 2020.
- [5] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.
- [6] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [7] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

A File structure

Dataset structure is shown as below

```
datasets/  
test/  
val/  
cat/  
dog/  
train/  
cat/  
dog/  
train_aug/  
cat/  
dog/
```

B Code

File: VGG-16.py

```
1 import os  
2 import zipfile  
3 import pandas as pd  
4 import numpy as np  
5 import cv2  
6 import matplotlib  
7 matplotlib.use('tkagg')  
8  
9 # Visual  
10 import seaborn as sns  
11 from PIL import Image  
12 import cv2  
13 from matplotlib.image import imread  
14 import matplotlib.pyplot as plt  
15  
16 # Scikit-learn  
17 from sklearn.metrics import classification_report, confusion_matrix,  
    ConfusionMatrixDisplay  
18 from sklearn.model_selection import train_test_split  
19  
20 # Tensorflow  
21 import tensorflow as tf  
22 from tensorflow.keras.preprocessing.image import ImageDataGenerator  
23 from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping  
24 from keras.applications import VGG16  
25 from keras.applications.vgg16 import preprocess_input  
26 from keras.models import Sequential, Model
```

```

27 from keras.layers import Flatten,Dense,Dropout
28
29 batch_size = 64
30 size = (370, 370)
31
32 # Create image data generator
33 idg = tf.keras.preprocessing.image.ImageDataGenerator(
    preprocessing_function = tf.keras.applications.vgg16.preprocess_input)
34
35 train_idg = idg.flow_from_dataframe(data, "datasets/train/", x_col= "
    filename", y_col= "label",
36                                     batch_size = batch_size,
37                                     target_size=size, validate_filenames=
    False)
38
39 val_idg = idg.flow_from_dataframe(data_val, "datasets/val/", x_col="
    filename", y_col="label",
40                                     batch_size = batch_size,
41                                     target_size = size, shuffle = False,
    validate_filenames=False)
42
43 test_idg = idg.flow_from_dataframe(data_test, "datasets/test/", x_col="
    filename", y_col="label",
44                                     batch_size = batch_size,
45                                     target_size = size, shuffle = False,
    validate_filenames=False)
46
47 vgg16_model = tf.keras.applications.vgg16.VGG16(include_top=False,
    input_shape=(370, 370, 3))
48
49 # Freeze Layers
50 for layer in vgg16_model.layers:
51     layer.trainable = False
52 # Unfreeze last five layers
53 flat = tf.keras.layers.Flatten() (vgg16_model.output)
54 dropout1 = tf.keras.layers.Dropout(0.2, name="Dropout1") (flat)
55 dense1 = tf.keras.layers.Dense(128, activation="relu") (dropout1)
56 dropout2 = tf.keras.layers.Dropout(0.2, name="Dropout2") (dense1)
57 output = tf.keras.layers.Dense(2, activation="softmax") (dropout2)
58 final_model = tf.keras.models.Model(inputs=[vgg16_model.input], outputs=[
    output])
59
60 # Instantiate model with Adam optimizer and Categorical loss entropy
61 final_model.compile(optimizer='adam',loss=tf.keras.losses.
    categorical_crossentropy,metrics = ["acc"])
62
63 # Set callbacks
64 learning_rate_reduction = ReduceLRonPlateau(monitor = 'val_acc', patience
    =2,

```

```

65         factor=0.5, min_lr = 0.00001,
66         verbose = 1)
67
68 early_stopping = EarlyStopping(monitor='val_loss', patience=5,
69                                restore_best_weights=True, verbose=0)
70
71 # Train Model
72 history = final_model.fit(train_idg,
73                            batch_size=batch_size,
74                            validation_data=val_idg,
75                            epochs = 20,
76                            callbacks=[learning_rate_reduction,early_stopping
77                                     ])
78
79 # Run model on test data
80 result = final_model.predict(test_idg)
81
82 result_argmax = np.argmax(result, axis=1)
83
84 # Retraining model on CIFAR-10 dataset
85 from keras.datasets import cifar10
86
87 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
88 assert x_train.shape == (50000, 32, 32, 3)
89 assert x_test.shape == (10000, 32, 32, 3)
90 assert y_train.shape == (50000, 1)
91 assert y_test.shape == (10000, 1)
92
93 x_test, x_val,y_test, y_val = train_test_split(x_test, y_test, test_size
94         =0.5, random_state = 23)
95
96 from tensorflow.keras.utils import to_categorical
97
98 # Convert labels to one-hot encoding
99 y_train = to_categorical(y_train, num_classes=10)
100 y_val = to_categorical(y_val, num_classes=10)
101
102 vgg16_model_cifar = tf.keras.applications.vgg16.VGG16(include_top=False,
103         input_shape=(32, 32, 3))
104
105 for layer in vgg16_model_cifar.layers:
106     layer.trainable = False
107
108 flat = tf.keras.layers.Flatten() (vgg16_model_cifar.output)
109 dropout1 = tf.keras.layers.Dropout(0.2, name="Dropout1") (flat)
110 densel = tf.keras.layers.Dense(128, activation="relu") (dropout1)
111 dropout2 = tf.keras.layers.Dropout(0.2, name="Dropout2") (densel)
112 output = tf.keras.layers.Dense(10, activation="softmax") (dropout2)

```

```

111 final_model_cifar = tf.keras.models.Model(inputs=[vgg16_model_cifar.input],
      outputs=[output])
112 final_model_cifar.compile(optimizer='adam', loss=tf.keras.losses.
      categorical_crossentropy, metrics = ["acc"])
113
114 learning_rate_reduction = ReduceLROnPlateau(monitor = 'val_acc', patience
      =2,
115                                             factor=0.5, min_lr = 0.00001,
116                                             verbose = 1)
117
118 early_stopping = EarlyStopping(monitor='val_loss', patience=5,
      restore_best_weights=True, verbose=1)
119
120
121 history_cifar = final_model_cifar.fit(x_train, y_train,
      batch_size=batch_size,
122                                     validation_data=(x_val,y_val),
123                                     epochs = 100,
124                                     callbacks=[learning_rate_reduction,early_stopping
125                                             ])
126
127 result_cifar = final_model_cifar.predict(x_test)
128
129 result_cifar_argmax = np.argmax(result_cifar, axis=1)

```

Listing 1: Data augmentation

File: data_aug.py

```

1 import random
2 import os
3 import shutil
4 import cv2
5 from tqdm import tqdm
6 import albumentations as A
7
8 dog_dir = './datasets/train/dog/'
9 aug_dog_dir = './datasets/train_aug/dog/'
10 cat_dir = './datasets/train/cat/'
11 aug_cat_dir = './datasets/train_aug/cat/'
12 os.makedirs(aug_dog_dir, exist_ok=True)
13 os.makedirs(aug_cat_dir, exist_ok=True)
14
15 random.seed(10)
16
17 transform = A.Compose([
18     A.ChannelShuffle(), # RGB
19     A.HueSaturationValue(), # HSV
20     A.RandomRotate90(), # 90

```



```

21     A.ShiftScaleRotate(shift_limit=0.075, scale_limit=0.45, rotate_limit
    =45, p=0.75), #
22     A.Blur(blur_limit=5, p=0.5), #
23     A.RandomBrightnessContrast(p=0.5), #
24     A.HorizontalFlip(p=0.5), #
25     A.VerticalFlip(p=0.5) #
26 ])
27
28 for file in tqdm(os.listdir(dog_dir)):
29     image = cv2.imread(os.path.join(dog_dir, file))
30     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
31     augmented_image = transform(image=image)['image']
32     cv2.imwrite(os.path.join(aug_dog_dir, 'aug_'+file), cv2.cvtColor(
        augmented_image, cv2.COLOR_RGB2BGR))
33     shutil.copy(os.path.join(dog_dir, file), os.path.join(aug_dog_dir, file
        ))
34
35 for file in tqdm(os.listdir(cat_dir)):
36     image = cv2.imread(os.path.join(cat_dir, file))
37     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
38     augmented_image = transform(image=image)['image']
39     cv2.imwrite(os.path.join(aug_cat_dir, 'aug_'+file), cv2.cvtColor(
        augmented_image, cv2.COLOR_RGB2BGR))
40     shutil.copy(os.path.join(cat_dir, file), os.path.join(aug_cat_dir, file
        ))
41
42 print('CAT training data after augmentation', len(os.listdir('
    datasets/train_aug/cat/')))
43 print('DOG training data after augmentation', len(os.listdir('
    datasets/train_aug/dog/')))

```

Listing 2: Data augmentation

File: resnet50_mod.py

```

1 import os
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 from tqdm import tqdm
5 import pandas as pd
6
7
8 img_height, img_width = 128, 128
9 batch_size = 32
10 train_dir = "./datasets"
11
12 train_ds = tf.keras.utils.image_dataset_from_directory(
13     os.path.join(train_dir, "train_aug"),
14     image_size=(img_height, img_width),

```

```

15     batch_size=batch_size
16 )
17
18 val_ds = tf.keras.utils.image_dataset_from_directory(
19     os.path.join(train_dir, "val"),
20     image_size=(img_height, img_width),
21     batch_size=batch_size
22 )
23
24 model = tf.keras.Sequential()
25
26 model.add(tf.keras.layers.Rescaling(1./255))
27 model.add(tf.keras.applications.ResNet50(include_top=False, pooling='max',
28     weights='imagenet'))
29 model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
30
31 model.compile(
32     optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
33     loss=tf.keras.losses.BinaryCrossentropy(),
34     metrics=["accuracy"]
35 )
36
37 earlyStopping = tf.keras.callbacks.EarlyStopping(monitor="val_loss",
38     patience=10, verbose=1)
39
40 EPOCHS = 50
41
42 history = model.fit(
43     train_ds,
44     validation_data=val_ds,
45     epochs=EPOCHS,
46     callbacks=[earlyStopping]
47 )
48
49 model.summary()
50
51 fig1 = plt.gcf()
52 plt.plot(history.history['accuracy'])
53 plt.plot(history.history['val_accuracy'])
54 plt.axis(ymin=0.4, ymax=1)
55 plt.grid()
56 plt.title('Model Accuracy')
57 plt.ylabel('Accuracy')
58 plt.xlabel('Epochs')
59 plt.legend(['train', 'validation'])
60 plt.show()
61 model.evaluate(val_ds)
62
63 fig2 = plt.gcf()

```

```

62 plt.plot(history.history['loss'])
63 plt.plot(history.history['val_loss'])
64 plt.axis(ymin=0,ymax=1)
65 plt.grid()
66 plt.title('Model Loss')
67 plt.ylabel('Accuracy')
68 plt.xlabel('Epochs')
69 plt.legend(['train', 'validation'])
70 plt.show()

```

Listing 3: Resnet50 train and validation

File: ViT_model.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class PositionEmbs(nn.Module):
6     def __init__(self, num_patches, emb_dim, dropout_rate = 0.1):
7         super(PositionEmbs, self).__init__()
8         self.pos_embedding = nn.Parameter(torch.randn(1, num_patches + 1,
9             emb_dim))
10        if dropout_rate > 0:
11            self.dropout = nn.Dropout(dropout_rate)
12        else:
13            self.dropout = None
14
15        def forward(self, x):
16            out = x + self.pos_embedding
17
18            if self.dropout:
19                out = self.dropout(out)
20
21            return out
22
23 class MlpBlock(nn.Module):
24     """ Transformer Feed-Forward Block """
25     def __init__(self, in_dim, mlp_dim, out_dim, dropout_rate = 0.1):
26         super(MlpBlock, self).__init__()
27         # init layers
28         self.fc1 = nn.Linear(in_dim, mlp_dim)
29         self.fc2 = nn.Linear(mlp_dim, out_dim)
30         self.act = nn.GELU()
31         if dropout_rate > 0.0:
32             self.dropout1 = nn.Dropout(dropout_rate)
33             self.dropout2 = nn.Dropout(dropout_rate)
34         else:
35             self.dropout1 = None

```

```

35         self.dropout2 = None
36
37     def forward(self, x):
38         out = self.fc1(x)
39         out = self.act(out)
40         if self.dropout1:
41             out = self.dropout1(out)
42         out = self.fc2(out)
43         out = self.dropout2(out)
44         return out
45
46 class LinearGeneral(nn.Module):
47     def __init__(self, in_dim = (768,), feat_dim = (12, 64)):
48         super(LinearGeneral, self).__init__()
49
50         self.weight = nn.Parameter(torch.randn(*in_dim, *feat_dim))
51         self.bias = nn.Parameter(torch.zeros(*feat_dim))
52
53     def forward(self, x, dims):
54         a = torch.tensordot(x, self.weight, dims = dims) + self.bias
55         return a
56
57
58 class SelfAttention(nn.Module):
59     def __init__(self, in_dim, heads = 8, dropout_rate = 0.1):
60         super(SelfAttention, self).__init__()
61         self.heads = heads
62         self.head_dim = in_dim // heads
63         self.scale = self.head_dim ** 0.5
64
65         self.query = LinearGeneral((in_dim,), (self.heads, self.head_dim))
66         self.key = LinearGeneral((in_dim,), (self.heads, self.head_dim))
67         self.value = LinearGeneral((in_dim,), (self.heads, self.head_dim))
68         self.out = LinearGeneral((self.heads, self.head_dim), (in_dim,))
69
70         if dropout_rate > 0:
71             self.dropout = nn.Dropout(dropout_rate)
72         else:
73             self.dropout = None
74
75     def forward(self, x):
76         b, n, _ = x.shape
77
78         q = self.query(x, dims = ([2], [0]))
79         k = self.key(x, dims = ([2], [0]))
80         v = self.value(x, dims = ([2], [0]))
81
82         q = q.permute(0, 2, 1, 3)
83         k = k.permute(0, 2, 1, 3)

```

```

84     v = v.permute(0, 2, 1, 3)
85
86     attn_weights = torch.matmul(q, k.transpose(-2, -1)) / self.scale
87     attn_weights = F.softmax(attn_weights, dim = -1)
88     out = torch.matmul(attn_weights, v)
89     out = out.permute(0, 2, 1, 3)
90     out = self.out(out, dims = ([2, 3], [0, 1]))
91     return out
92
93
94 class EncoderBlock(nn.Module):
95     def __init__(self, in_dim, mlp_dim, num_heads, dropout_rate = 0.1,
96                 attn_dropout_rate = 0.1):
97         super(EncoderBlock, self).__init__()
98
99         self.norm1 = nn.LayerNorm(in_dim)
100        self.attn = SelfAttention(in_dim, heads = num_heads, dropout_rate =
101                                attn_dropout_rate)
102        if dropout_rate > 0:
103            self.dropout = nn.Dropout(dropout_rate)
104        else:
105            self.dropout = None
106        self.norm2 = nn.LayerNorm(in_dim)
107        self.mlp = MlpBlock(in_dim, mlp_dim, in_dim, dropout_rate)
108
109    def forward(self, x):
110        residual = x
111        out = self.norm1(x)
112        out = self.attn(out)
113        if self.dropout:
114            out = self.dropout(out)
115        out += residual
116        residual = out
117        out = self.norm2(out)
118        out = self.mlp(out)
119        out += residual
120        return out
121
122 class Encoder(nn.Module):
123     def __init__(self, num_patches, emb_dim, mlp_dim, num_layers = 12,
124                 num_heads = 12, dropout_rate = 0.1, attn_dropout_rate = 0.0):
125         super(Encoder, self).__init__()
126         # positional embedding
127         self.pos_embedding = PositionEmbs(num_patches, emb_dim,
128                                           dropout_rate)
129         # encoder blocks
130         in_dim = emb_dim
131         self.encoder_layers = nn.ModuleList()

```

```

129         for _ in range(num_layers):
130             layer = EncoderBlock(in_dim, mlp_dim, num_heads, dropout_rate,
131                                   attn_dropout_rate)
132             self.encoder_layers.append(layer)
133         self.norm = nn.LayerNorm(in_dim)
134
135     def forward(self, x):
136         out = self.pos_embedding(x)
137         for layer in self.encoder_layers:
138             out = layer(out)
139         out = self.norm(out)
140         return out
141
142 class VisionTransformer(nn.Module):
143     """ Vision Transformer """
144     def __init__(self,
145                 image_size = (256, 256),
146                 patch_size = (16, 16),
147                 emb_dim = 768,
148                 mlp_dim = 3072,
149                 num_heads = 12,
150                 num_layers = 12,
151                 attn_dropout_rate = 0.0,
152                 dropout_rate = 0.1):
153         super(VisionTransformer, self).__init__()
154         h, w = image_size
155
156         # embedding layer
157         fh, fw = patch_size
158         gh, gw = h // fh, w // fw
159         num_patches = gh * gw
160         self.embedding = nn.Conv2d(3, emb_dim, kernel_size = (fh, fw),
161                                     stride = (fh, fw))
162         # class token
163         self.cls_token = nn.Parameter(torch.zeros(1, 1, emb_dim))
164         # transformer
165         self.transformer = Encoder(
166             num_patches = num_patches,
167             emb_dim = emb_dim,
168             mlp_dim = mlp_dim,
169             num_layers = num_layers,
170             num_heads = num_heads,
171             dropout_rate = dropout_rate,
172             attn_dropout_rate = attn_dropout_rate)
173
174     def forward(self, x):
175         emb = self.embedding(x) # (n, c, gh, gw)
176         emb = emb.permute(0, 2, 3, 1) # (n, gh, hw, c)

```

```

176         b, h, w, c = emb.shape
177         emb = emb.reshape(b, h * w, c)
178         # prepend class token
179         cls_token = self.cls_token.repeat(b, 1, 1)
180         emb = torch.cat([cls_token, emb], dim = 1)
181         # transformer
182         feat = self.transformer(emb)
183         return feat
184
185     class CAFIA_Transformer(nn.Module):
186         def __init__(self, args):
187             super(CAFIA_Transformer, self).__init__()
188             self.vit = VisionTransformer(
189                 image_size = (args.image_size, args.image_size),
190                 patch_size = (args.patch_size, args.patch_size),
191                 emb_dim = args.emb_dim,
192                 mlp_dim = args.mlp_dim,
193                 num_heads = args.num_heads,
194                 num_layers = args.num_layers,
195                 attn_dropout_rate = args.attn_dropout_rate,
196                 dropout_rate = args.dropout_rate)
197             self.init_weight(args)
198             self.classifier = nn.Linear(args.emb_dim, args.num_classes)
199
200
201         def init_weight(self, args):
202             state_dict = torch.load(args.vit_model)['state_dict']
203             del state_dict['classifier.weight']
204             del state_dict['classifier.bias']
205             self.vit.load_state_dict(state_dict)
206
207         def forward(self, batch_X):
208             feat = self.vit(batch_X)
209             output = self.classifier(feat[:, 0])
210             return output

```

Listing 4: Vision Transformer model

File: ViT_train.py

```

1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import torch.nn.functional as F
6 from model import CAFIA_Transformer
7 from tqdm import tqdm
8 import logging
9 from scheduler import cosine_lr

```

```

10 logging.basicConfig(level = logging.NOTSET)
11 os.environ["CUDA_VISIBLE_DEVICES"] = "0"
12 device = torch.device("cuda")
13
14 def train_model(args, trainloader, testloader):
15     n_gpu = 1
16     model = CAFIA_Transformer(args)
17     model.to(device)
18
19     if n_gpu > 1:
20         model = torch.nn.DataParallel(model)
21     model.train()
22     loss_fn = nn.CrossEntropyLoss()
23     optimizer = optim.AdamW(model.parameters(), betas=(args.betal, args.
        beta2), eps = args.eps, lr = args.learning_rate, weight_decay = args
        .weight_decay)
24
25     total_steps = (len(trainloader) // args.batch_size + 1) * args.epochs
26     scheduler = cosine_lr(optimizer, args.learning_rate, args.warmup,
        total_steps)
27     nb_tr_steps = 0
28     logging.info('***** start to train
        *****')
29     for epoch in range(args.epochs):
30         train_loss = 0
31         train_iter = 0
32         for i, batch in enumerate(tqdm(trainloader, desc = "Iteration")):
33             nb_tr_steps += 1
34             optimizer.zero_grad()
35             batch = tuple(t.to(device) for t in batch)
36             batch_X, batch_Y = batch
37             outputs = model(batch_X)
38             loss = loss_fn(outputs, batch_Y)
39             loss.backward()
40             optimizer.step()
41             scheduler(nb_tr_steps)
42             train_loss += loss.item()
43             train_iter += 1
44
45             logging.info('Epoch:%d batch_loss:%f', epoch, loss)
46
47         train_loss = loss / train_iter
48
49         #eval
50         logging.info('***** start to evaluate
            *****')
51         model.eval()
52         total, correct = 0, 0
53         for _, batch in enumerate(tqdm(testloader, desc = "Iteration")):

```



```

54         batch = tuple(t.to(device) for t in batch)
55         batch_X, batch_Y = batch
56         outputs = model(batch_X)
57         _, predicted = torch.max(outputs.data, 1)
58         total += batch_Y.size(0)
59         correct += (predicted == batch_Y).sum()
60
61     acc = (correct / total).item()
62     logging.info('Epoch: %d train_loss: %f Accuracy: %f', epoch,
63                 train_loss, acc)
64     if not os.path.exists(args.output):
65         os.mkdir(args.output)
66     output_path = os.path.join(args.output, str(acc)+'.pt')
67     torch.save(model.state_dict(), output_path)

```

Listing 5: Vision Transformer train and validation